SHL Assessment Recommender - Approach Document

Approach Document — SHL Assessment Recommender

Design choices

Catalog. The provided JSON endpoint returns 377 raw entries mixing 370 Individual Test Solutions with 7 pre-packaged Job Solutions (all literally named "`<Role> Solution`"). I filtered on that naming convention rather than a missing category field, since SHL's own scrape didn't tag solution type explicitly. The cleaned catalog is bundled with the app (`data/shl_catalog.json`) so the service never fails to start due to an external dependency; at startup it also attempts a live refresh from SHL's endpoint and silently falls back to the bundle on any error (I hit a 403 from my dev sandbox's IP when testing this — likely bot protection on SHL's side — so the fallback path matters more than the happy path here).

Retrieval: BM25, not embeddings. With only ~370 items and a highly domain-specific, often acronym-heavy vocabulary ("OPQ32r", "GSA", "DSI"), lexical search does well and avoids an embeddings API call (extra latency and cost per turn) or standing up a vector DB for what's really a small lookup table. I built a lightweight BM25 index (`rank_bm25`) over name+description+category text, combined with structured pre-filters (test type code, job level, max duration, language, remote-only) applied before ranking. Tested against representative single-topic queries (java, sql, excel, OPQ32r, cognitive ability, situational judgement) with strong top-3 precision. Composite queries ("java developer with stakeholder communication") rank worse when thrown at BM25 as one blob — but the agent design sidesteps this by encouraging multiple, narrower `search_catalog` calls per distinct requirement (mirroring how sample trace C9 builds a battery skill-by-skill), rather than relying on one search to do everything.

Agent: tool-calling loop, not a hardcoded state machine. I initially considered a rule-based intent classifier (clarify/ recommend/refine/compare/ refuse) driving separate prompts per state. I rejected it: the traces show these behaviors blending within a single turn (C7 refuses a legal question *and* keeps the existing shortlist in the same reply). Instead, Gemini 2.5 Flash gets three tools — `search_catalog`, `get_item_details`, and a mandatory `respond` — and a system prompt encoding the four behaviors, scope limits, and prompt-injection resistance, with the model deciding which tool(s) to invoke per turn. This handles blended behaviors naturally.

Hallucination guard (the part I spent the most time on). `respond` only accepts *identifiers* (catalog id or exact name) for its recommendations, never free-text names/URLs. The server resolves each identifier against the real catalog dict and drops anything that doesn't resolve — the model has no path to inject a fabricated name or URL into a recommendation. The markdown table shown to the user is also built server-side from the resolved rows, not written by the model, so displayed data is always exactly what's in the catalog. This also solves state continuity for a stateless API: since the client only sends back plain `role`/`content` history (no structured recs field, per the given schema), the server-built table embedded in the assistant's own prior `reply` text is what lets the model "remember" the current shortlist for refine/compare turns in later requests.

Turn budget. The evaluator caps conversations at 8 turns. The system prompt is given the current turn number and is told, from turn 3 onward, to commit to a best-effort recommendation with reasonable defaults rather than keep asking questions — otherwise a cautious agent could clarify forever and never produce a shortlist, failing every hard eval and the whole recall metric for that trace.

Prompt design

The system prompt is rebuilt per request (stateless) with: scope rules (SHL assessments only; explicit prompt-injection resistance instructing the model to treat embedded instructions in user text as untrusted content, not commands); the four behaviors with a strict grounding rule ("never state a fact about an assessment you didn't retrieve via a tool this conversation"); a formatting rule (no manual tables/URLs — the platform attaches those); and the dynamic turn-budget note. Full text is in `app/agent.py:SYSTEM_PROMPT_TEMPLATE`.

Evaluation approach

Unit-level: mocked the LLM client to test the tool-calling loop mechanics, the hallucination guard (verified fake/mixed identifiers get dropped, real ones survive), and the full HTTP round-trip via FastAPI's TestClient — all passing (see repo history / can be re-run on request).

Trace-level: `scripts/eval_harness.py` replays each of the 10 provided sample traces' *user* turns (not the canned agent replies) against the real running agent, and scores Recall@10 against the union of every URL mentioned across that trace as a proxy ground truth. I was unable to run this against the live Gemini API myself — my execution sandbox's network egress is restricted to a package-registry allowlist that doesn't include `generativelanguage.googleapis.com`, and I don't have a way around that from within the sandbox. **This is the one part of the submission that still needs a live run with a real API key** before deploying with full confidence; the harness and hallucination guard exist specifically so that run is fast to do and safe even if imperfect.

What didn't work / was reconsidered

- **Vector store for retrieval**: overkill for 370 rows; would have added an embeddings API call per turn and infra for no measurable retrieval benefit at this scale.
- **Rule-based conversation state machine**: rejected once the traces showed blended behaviors (refuse + keep shortlist) within one turn — a single LLM decision per turn with tools handles this more naturally than routing between separate prompts.
- **Trusting the LLM's own markdown table**: early design let the model write out the recommendation table directly in `reply`. Switched to server-generated tables from resolved catalog rows after realizing this was the one place a hallucinated URL could still leak through even with the identifier-based `respond` tool.

AI tool usage disclosure

Built interactively with Claude (Anthropic) as a pair-programming/agentic coding assistant for scaffolding, the retrieval/agent implementation, and this document. Design decisions (BM25 vs. vector store, tool-calling architecture, hallucination-guard approach, turn-budget handling) were made deliberately and are defensible in review — not accepted as suggested without evaluation.